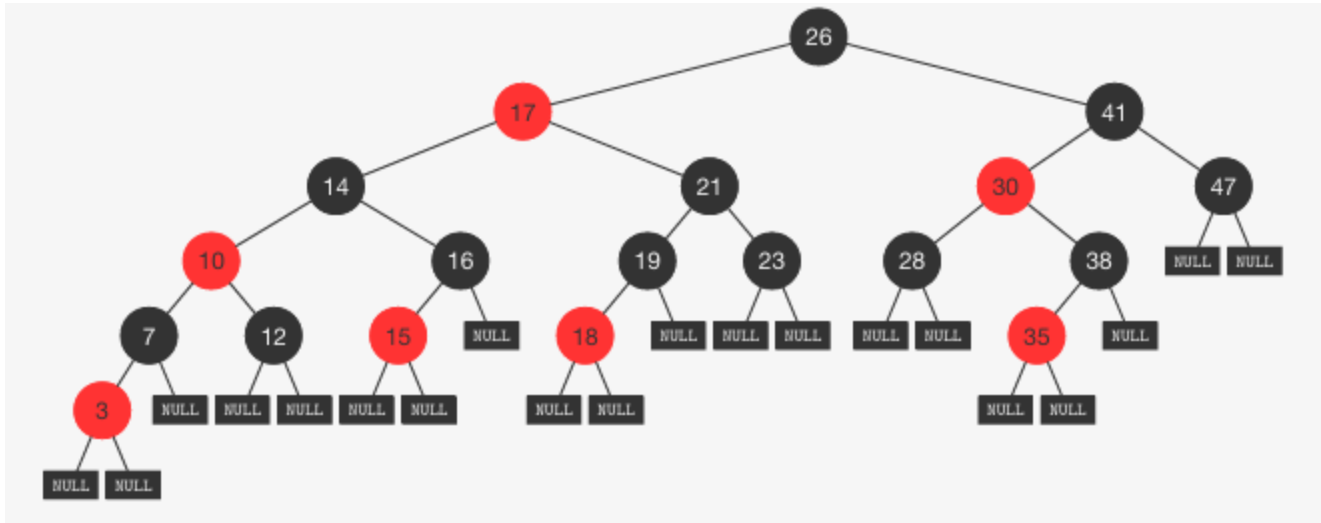


Árvore rubro-negra esquerdista



Uma **Árvore Binária de Busca (ABB)** Rubro-Negra Esquerdista é uma estrutura de dados que segue as seguintes propriedades:

1. **Cores dos Nós:**

- Cada nó da árvore é classificado como **vermelho** ou **preto**.

2. **Raiz Preta:**

- O nó raiz da árvore sempre tem a cor **preta**.

3. **Folhas Pretas:**

- Todas as folhas da árvore (representadas como nós **NULL**) são sempre **pretas**.

4. **Restrições para Nós Vermelhos:**

- Se um nó é vermelho, ele deve atender aos seguintes critérios:
 1. Ambos os seus filhos são **pretos**.
 2. Ele é sempre um filho esquerdo.

5. **Altura Negra:**

- Para qualquer nó, o número de nós **pretos** no caminho até qualquer folha descendente é o mesmo.
 1. O próprio nó não é contado.
 2. Este número constante é denominado **altura negra**.

Matematicamente, considerando bh com altura negra:

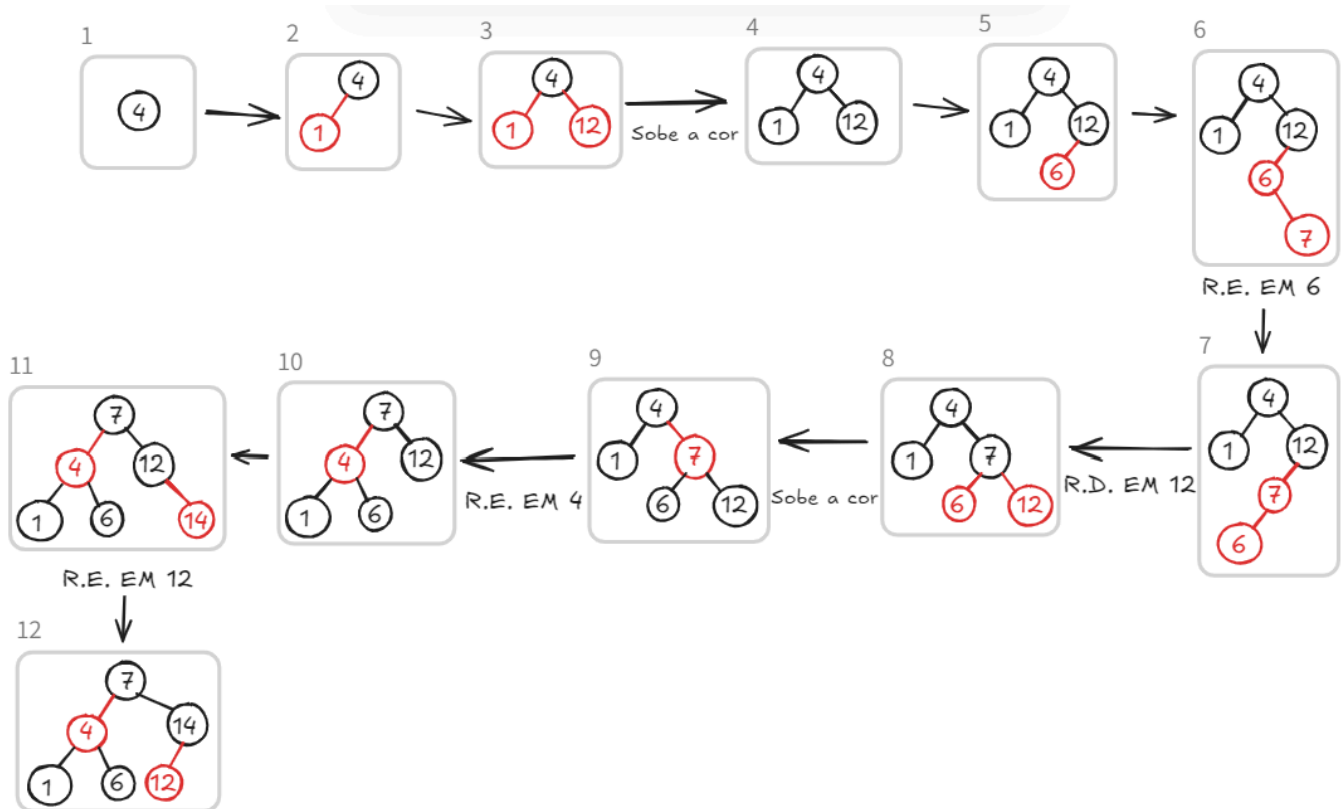
- O número de nós internos satisfaz: $n \geq 2^{bh} - 1 \geq 2^{h/2} - 1$
- A altura total é $O(\log n)$

Exemplo

Vamos inserir os seguintes números, na ordem dada:

4, 1, 12, 6, 7, 14

Passo a passo da construção



1. Inserindo o 4

1. Como ele é o primeiro nó, ele é preto

2. Inserindo o 1

1. Como 1 é menor que 4, ele se torna filho esquerdo vermelho

3. Inserindo 12

1. 12 é maior que 4, então vira filho direito

4. Como temos um filho esquerdo vermelho e um filho direito vermelho aplicamos um **sobe a cor** para manter o balanceamento

1. A raiz sempre precisa ser preta, então pintamos 4 de preto

5. Inserindo 6

2. Como o 6 é menor que 12, ele entra como filho esquerdo vermelho de 12

6. Inserindo 7

3. Como o 7 é maior que 6, ele entra como filho direito vermelho de 6

7. Como há um filho direito vermelho e o filho esquerdo é preto, realizamos uma rotação à esquerda em 6, promovendo 7 para o lugar de 6
 8. Como temos 2 filho esquerdos vermelhos seguidos fazemos uma rotação à direita em 12, fazendo com que:
 1. O 6 continue sendo filho esquerdo vermelho de 7
 2. 7 se torne preto
 3. 12 se torne o filho direito vermelho de 7
 9. Depois aplicamos o `sobe a cor` tornando 7 vermelho e os filhos do 7 pretos
 10. Depois rotacionamos em 4, fazendo com que:
 1. 1 continue sendo filho esquerdo de 4
 2. 6 vire filho direito preto de 4
 3. 4 se torne filho esquerdo vermelho de 7
 4. 12 continue sendo filho direito de 7
 11. Insere o 14
 2. Como 14 é maior que 12, ele entra como filho direito vermelho de 12
 12. Como há um filho direito vermelho e o filho esquerdo é preto, realizamos uma rotação à esquerda em 12, fazendo com que:
 1. 12 vire filho esquerdo vermelho de 14
 2. 14 vire o filho direito preto de 7
-

Implementação em C

Definição dos Nós

```
#include <stdio.h>
#include <stdlib.h>

typedef enum Cor {VERM, PRETO} Cor;

typedef struct No {
    int chave;
    Cor cor;
    struct No *esq, *dir;
} No;
```

Funções Auxiliares

Estas funções verificam a cor de um nó e ajudam a manter a estrutura balanceada.

```

int ehVerm(No *x){
    if(x != Null){
        return x->cor == VERM;
    }
    return 0;
}

int ehPreto(No *x){
    if(x != Null) return x->cor == PRETO;
    return 1;
}

```

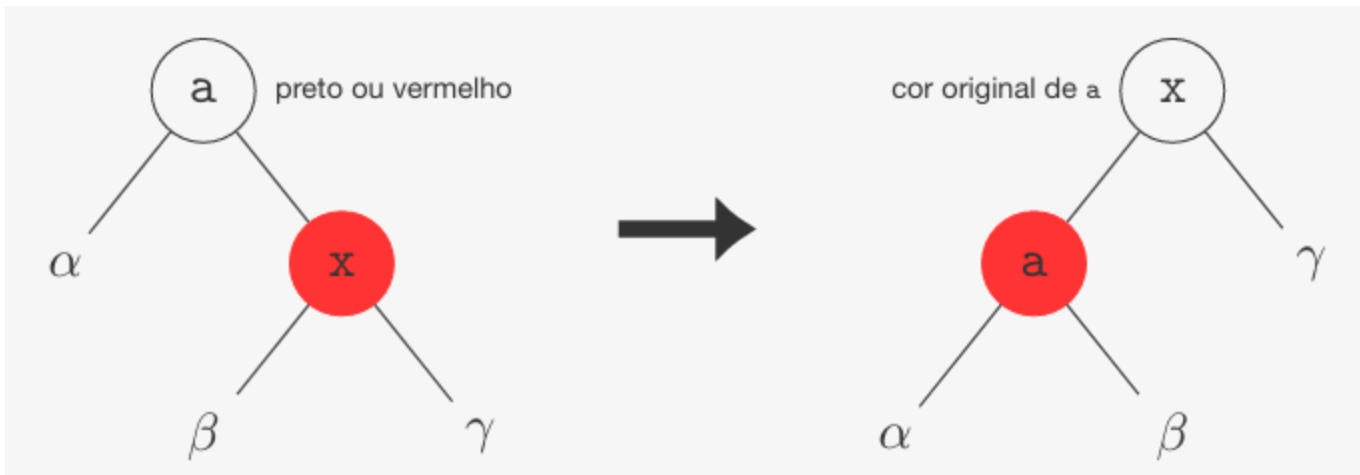
Operações Fundamentais

Rotações:

As rotações são usadas para manter o balanceamento da árvore.

1. Para Esquerda

Esta operação move um nó para a esquerda, promovendo seu filho direito.



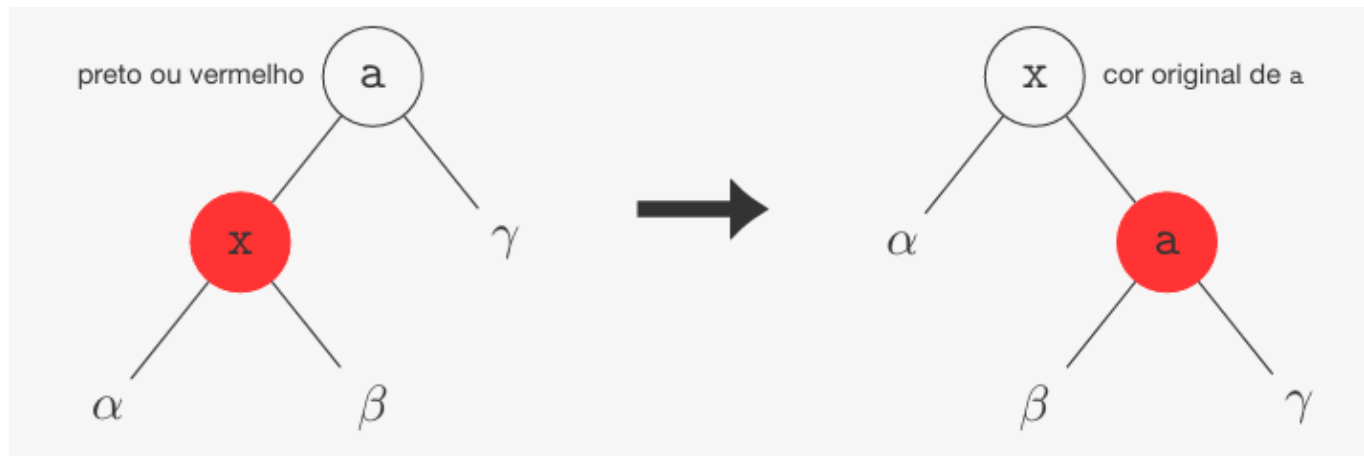
```

No* rot_esq(No* raiz){
    No* x = raiz->dir;
    raiz->dir = x->esq;
    x->esq = raiz;
    x->cor = raiz->cor;
    raiz->cor = VERM;
    return x;
}

```

2. Para Direita

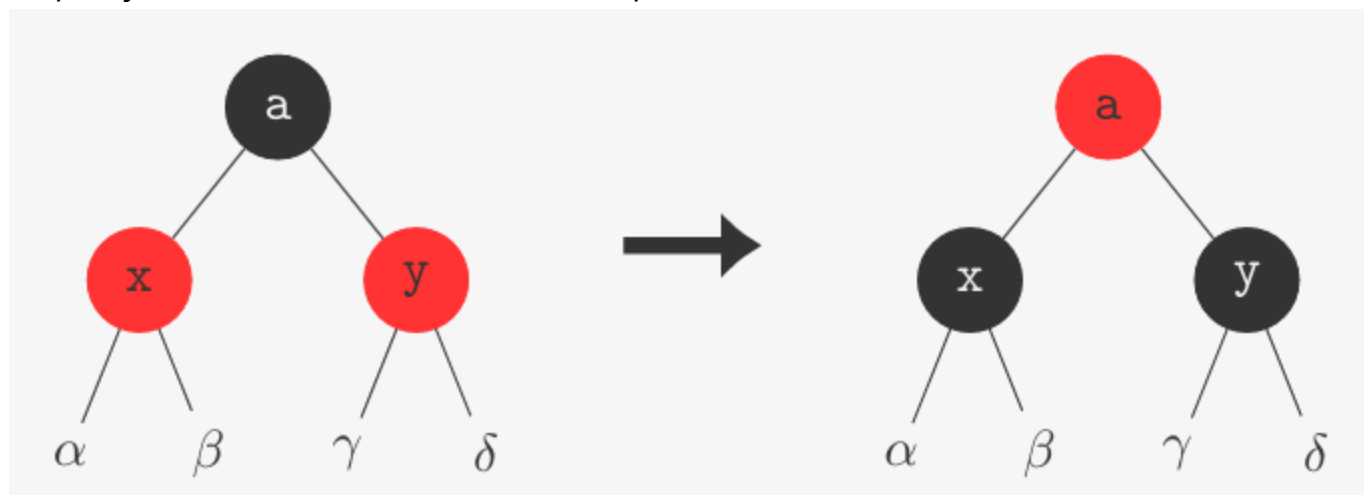
Esta operação move um nó para a direita, promovendo seu filho esquerdo.



```
No* rot_dir(No* raiz){  
    No* x = raiz->esq;  
    raiz->esq = x->dir;  
    x->dir = raiz;  
    x->cor = raiz->cor;  
    raiz->cor = VERM;  
    return x;  
}
```

Subida de Cor

A operação de subir cor redistribui as cores para manter o balanceamento.



```
void sobe_verm(No* raiz){  
    raiz->cor = VERM;  
    raiz->esq->cor = PRETO;  
    raiz->dir->cor = PRETO;  
}
```

Inserção na Árvore

A inserção segue as regras de uma árvore binária de busca, ajustando a árvore conforme necessário para preservar as propriedades da árvore rubro-negra esquerdista.

```
No *corrigir(No *r){
    if(ehPreto(r->esq) && ehVerm(r->dir)){
        r = rot_esq(r);
    }
    if(ehVerm(r->esq) && ehVerm(r->esq->esq)){
        r = rot_dir(r);
    }
    if(ehVerm(r->esq) && ehVerm(r->dir)){
        sobe_cor(r);
    }
    return r;
}
```

A função de inserção insere um novo nó e chama a função `corrigir` para restaurar o balanceamento.

```
No *insere(No *r, int x){
    if(r != NULL){
        if(x < r->chave) r->esq = insere(r->esq, x);
        else if(x > r->chave) r->dir = insere(r->dir, x);

        r = corrigir(r);
        return r;
    }else {
        No *novo = malloc(sizeof(No));
        novo->chave = x;
        novo->esq = novo->dir = NULL;
        novo->cor = VERM;
        return novo;
    }
}
```

Casos de Inserção

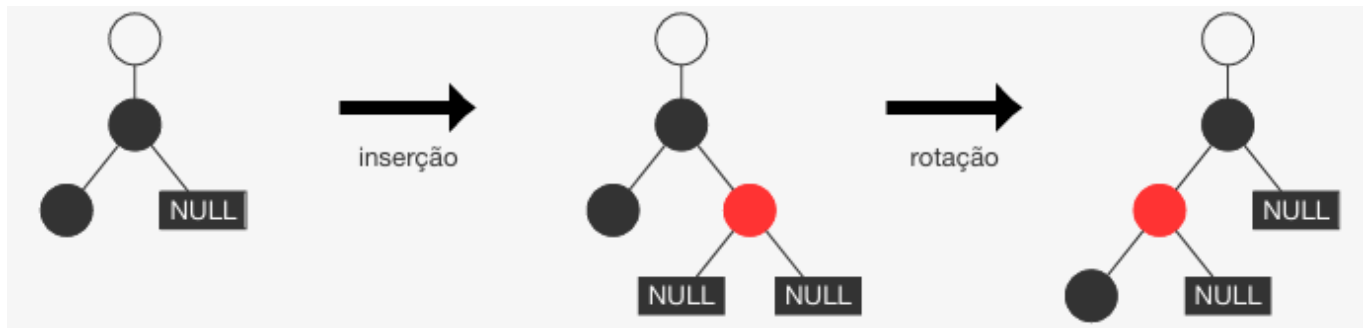
Caso 1: Inserindo no Filho Esquerdo

- Pai preto, filho direito preto

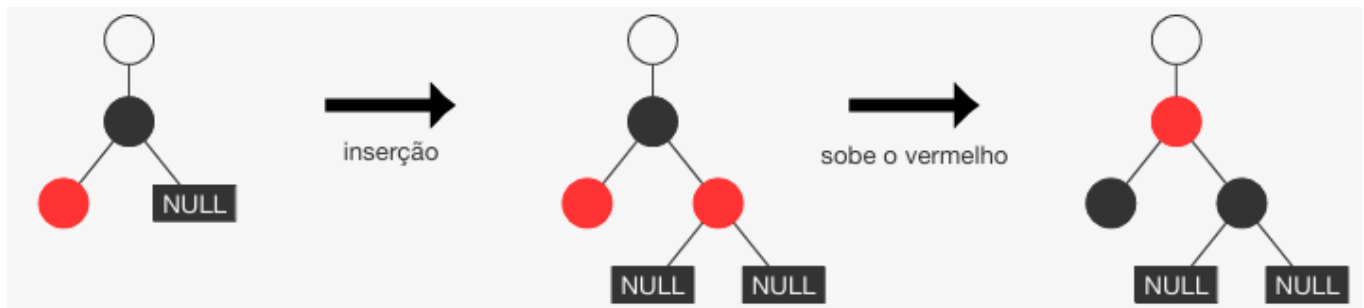


Caso 2: Inserindo no Filho Direito

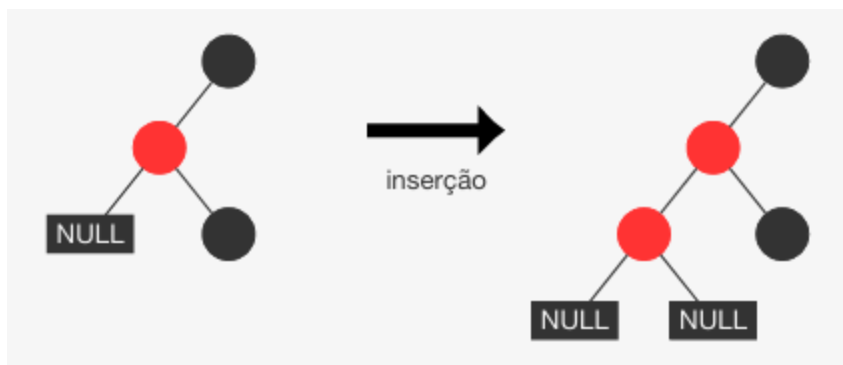
- Pai preto, filho esquerdo preto



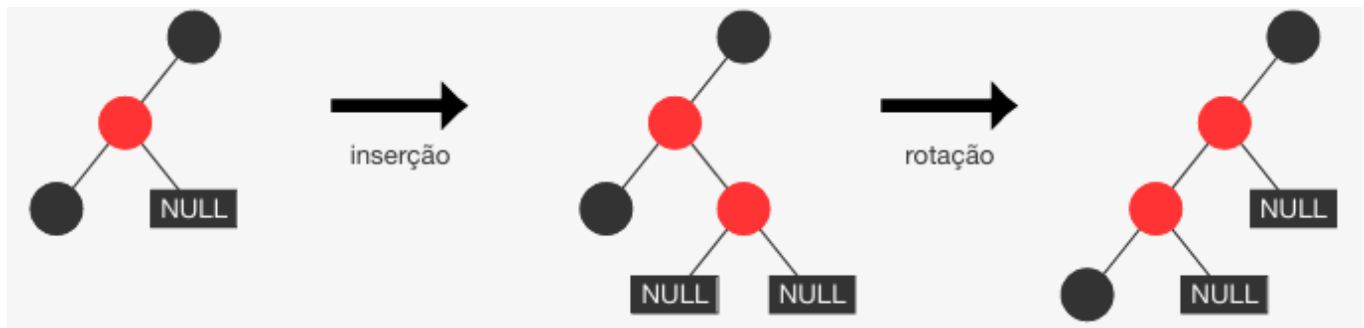
Caso 3: Pai vermelho, inserindo no Filho Direito



Caso 4: Pai preto, Inserindo no Filho Esquerdo



Caso 5: Pai preto, Inserindo no Filho Direito

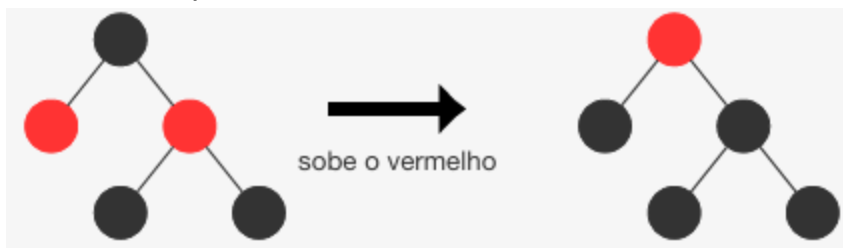


Corrigindo Problemas no Pai

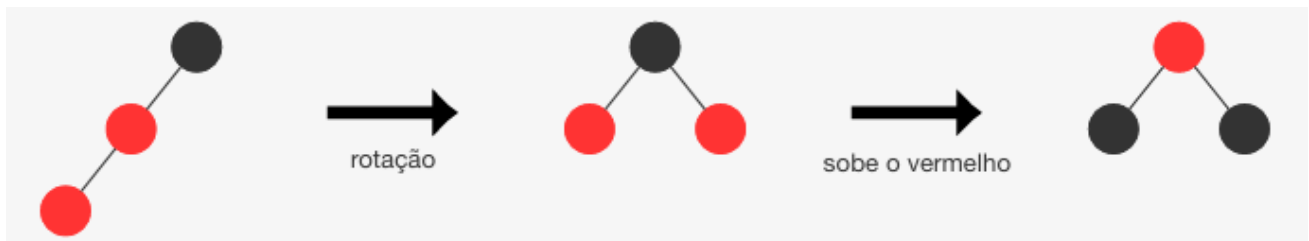
- Se o filho direito for vermelho, rotacionamos para esquerda



- Se o filho esquerdo for vermelho, basta subir a cor



- Se o filho esquerdo e seu neto esquerdo forem vermelhos, rotacionamos para a direita



Busca na Árvore

A busca em uma Árvore Rubro-Negra Esquerdista segue a lógica de uma **árvore binária de busca**.

```
No* busca(No *r, int x) {
    if (r == NULL || r->chave == x) {
        return r;
    }
}
```

```
}
if (x < r->chave) {
    return busca(r->esq, x);
} else {
    return busca(r->dir, x);
}
}
```

Remoção na Árvore

A remoção segue as regras de uma árvore binária de busca, garantindo que a árvore continue balanceada.

Mover Vermelho para Esquerda

Antes de remover um nó, garantimos que o **filho esquerdo** seja vermelho. Isso simplifica a remoção.

```
No* move_verm_esq(No* r) {
    sobe_verm(r);
    if (ehVerm(r->dir->esq)) {
        r->dir = rot_dir(r->dir);
        r = rot_esq(r);
        sobe_verm(r);
    }
    return r;
}
```

Mover vermelho para Direita

Se estamos tentando remover um nó do lado **direito**, precisamos garantir que há um nó vermelho no caminho.

```
No* move_verm_dir(No* r) {
    sobe_verm(r);
    if (ehVerm(r->esq->esq)) {
        r = rot_dir(r);
    }
    return r;
}
```

Remoção Mínima

Para remover um nó, podemos precisar encontrar o menor elemento (sucessor). Esta função remove o menor nó de uma subárvore.

```
No* remove_min(No* r) {
    if (r->esq == NULL) {
        free(r);
        return NULL;
    }
    if (ehPreto(r->esq) && ehPreto(r->esq->esq)) {
        r = move_verm_esq(r);
    }
    r->esq = remove_min(r->esq);
    return corrige(r);
}
```

Função Principal de Remoção

Essa é a função principal que remove um nó da árvore.

```
No* remove_no(No* r, int x) {
    if (x < r->chave) {
        if (ehPreto(r->esq) && ehPreto(r->esq->esq)) {
            r = move_verm_esq(r);
        }
        r->esq = remove_no(r->esq, x);
    } else {
        if (ehVerm(r->esq)) {
            r = rot_dir(r);
        }
        if (x == r->chave && r->dir == NULL) {
            free(r);
            return NULL;
        }
        if (ehPreto(r->dir) && ehPreto(r->dir->esq)) {
            r = move_verm_dir(r);
        }
        if (x == r->chave) {
            No* min = r->dir;
            while (min->esq != NULL) {
                min = min->esq;
            }
            r->chave = min->chave;
        }
    }
}
```

```
        r->dir = remove_min(r->dir);
    } else {
        r->dir = remove_no(r->dir, x);
    }
}
return corrige(r);
}
```

Conclusão

A **Árvore Rubro-Negra Esquerdista** é uma variação da árvore rubro-negra tradicional, oferecendo:

- **Busca, inserção e Remoção** eficientes em tempo $O(\log n)$.
- **Menos rotações**, simplificando a manutenção do balanceamento.

Este modelo é amplamente utilizado em estruturas de dados dinâmicas como tabelas de símbolos e banco de dados.